

202044502
Programming with
JAVA

Unit -5

Multi-Threading

Academic Year: 2025-26 (II)

Prepared By:
Prof. Pratik B. Soni
(Asst. Prof. – I.T. Dept, MBIT)

Multi-Threading -

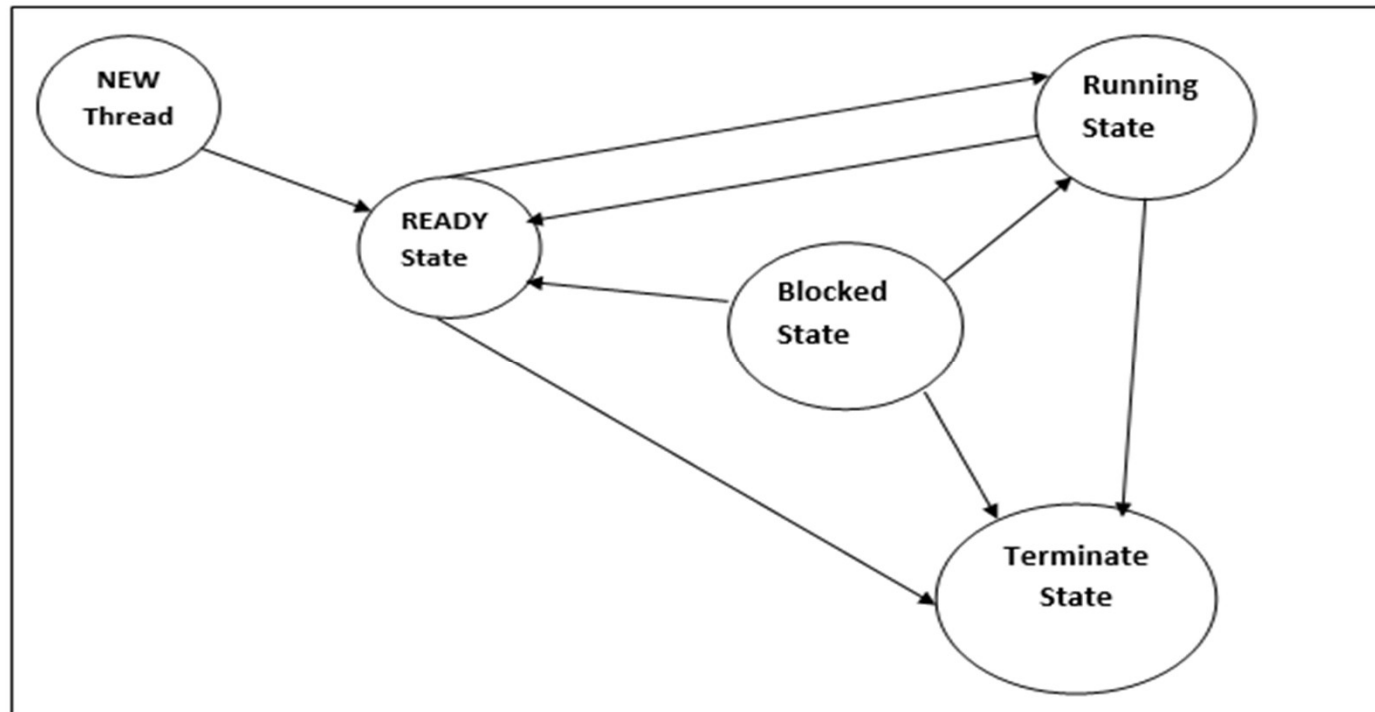
- Introduction
- Methods
- Main Thread
- Creating Threads
- Synchronization
- Thread Priorities

Multi-Threading -

- Introduction
- Methods
- Main Thread
- Creating Threads
- Synchronization
- Thread Priorities

Multi-Threading - Introduction

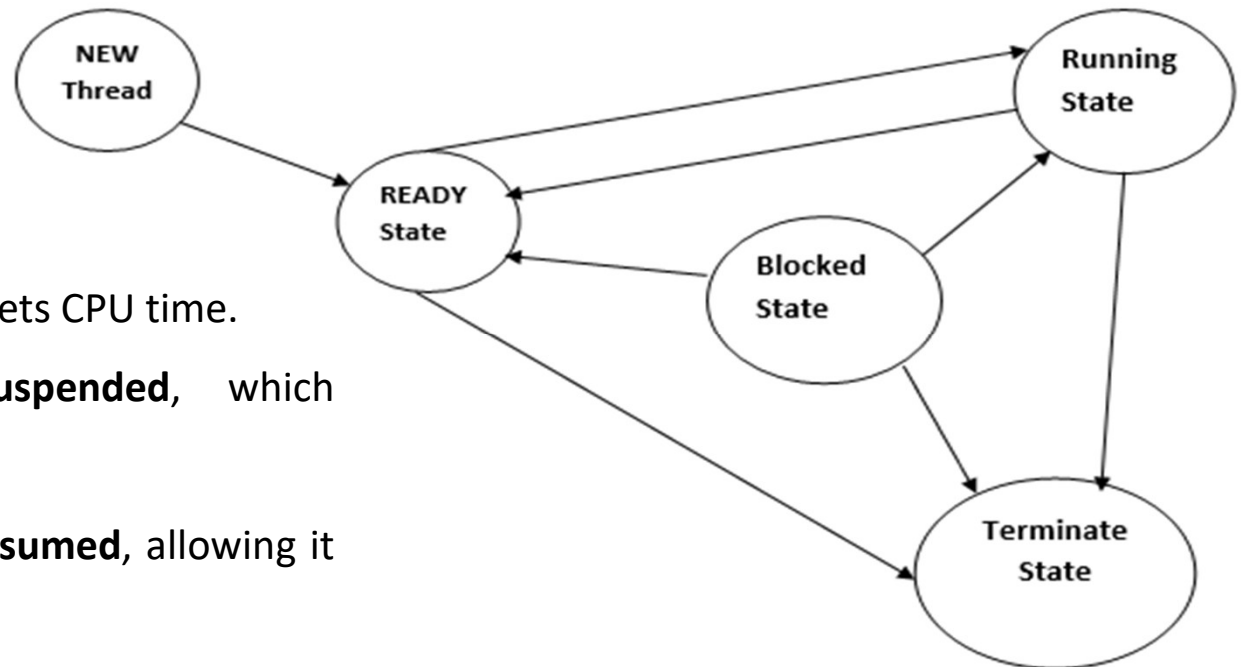
- The benefit of Java's multithreading is that the main loop/polling mechanism is eliminated.
- One thread can pause without stopping other parts of your program.
- Ultimately threading in JAVA provides different computations or executions at a single time concurrently.
- Thread has Several States.



Multi-Threading - Introduction

- Threads exist in several states.

- A thread can be **running**.
- It can be **ready to run** as soon as it gets CPU time.
- A running thread can be **suspended**, which temporarily suspends its activity.
- A suspended thread can then be **resumed**, allowing it to pick up where it left off.
- A thread can be blocked when **waiting** for a resource.
- At any time, a thread can be **terminated**, which halts its execution immediately. Once terminated, a thread cannot be resumed.



Multi-Threading - Introduction

Threads with Priorities

- Java assigns to each thread a priority that **determines how that thread should be treated** with respect to the others. Thread **priorities are integers** that specify the relative priority of one thread to another. a
- **As an absolute value, a priority is meaningless; *a higher-priority thread doesn't run any faster than a lower-priority thread if it is the only thread running.***
- Instead, a **thread's priority is used to decide when to switch from one running thread to the next.** This is called a **context switch.**

Multi-Threading -

- Introduction
- **Methods**
- Main Thread
- Creating Threads
- Synchronization
- Thread Priorities

Multi-Threading - Methods

<u>Method</u>	<u>Meaning</u>
getName ()	Obtain a thread's name.
getPriority ()	Obtain a thread's priority.
isAlive ()	Determine if a thread is still running.
join ()	Wait for a thread to terminate.
run ()	Entry point for the thread.
sleep ()	Suspend a thread for a period of time.
start ()	Start a thread by calling its run method
currentThread()	Returns current thread information

Multi-Threading -

- Introduction
- Methods
- **Main Thread**
- Creating Threads
- Synchronization
- Thread Priorities

Multi-Threading – Main Thread

- When a Java program starts up, **one thread begins running immediately**.
- **This is usually called the main thread of your program**, because it is the one that is executed when **program begins**.
- **The main thread is important for two reasons:**
 - 1) It is the thread from which **other “child” threads will be Created / Generated**.
 - 2) often it must be the **last thread to finish execution** because it performs various shutdown actions.

Multi-Threading – Main Thread (Thread_name, Priority, Thread_Group)

```
class data{
public static void main(String args[])
{
    Thread t = Thread.currentThread();
    System.out.println("Current thread: " + t);

    t.setName("My Thread");
    System.out.println("" +t);
    try {
        for(int n = 5; n > 0; n--)
        {
            System.out.println(n);
            Thread.sleep(1000);
        }
    } catch (InterruptedException e)
    {
        System.out.println("Main thread interrupted");
    }
} }
```

Output:

Current thread: Thread[main,5,main]

Thread[My Thread,5,main]

5

4

3

2

1

“Thread.sleep(1000); “ is used to sleep the thread for some time.

Here “1000” defines 1000 msec = 1 sec for thread sleeping.

It must be inside the “try-catch” block in program.

Multi-Threading -

- Introduction
- Methods
- Main Thread
- **Creating Threads**
- Synchronization
- Thread Priorities

Multi-Threading – Creating Threads

- Java defines two ways in which this can be accomplished:
 - (1)implement the Runnable interface.
 - (2)extend the Thread class, itself.

Multi-Threading – Creating Threads

- By implement the Runnable interface.

- Steps:

(1) Create class which implements the Runnable interface.

(2) Create constructor

i. Create “thread” object.

ii. Start the thread using thread object.

(3) Create “public void run()” method.

(4) In main() function, create object of the class in which thread is created.

- When the thread is started, at a time the “run()” method is called.
- So, actual logic or computation is available in the “run()” method.

- By Extending the Thread Class.

- Steps:

(1) Create class which extends “Thread” class.

(2) Create constructor

- *Using “super” keyword call the Thread class constructor (which creates the new thread)*

- *Start the thread.*

(3) Create “public void run()” method.

(4) In main() function, create object of the class in which thread is created.

- When the thread is started, at a time the “run()” method is called.
- So, actual logic or computation is available in the “run()” method.

Multi-Threading – Creating Threads

- By implement the Runnable interface.

```
class t1 implements Runnable{
    Thread t;
    String name;
    t1() {
        t = new Thread(this);
        System.out.println("t:"+t);
        t.start();
    }
    public void run(){
        try{
            for(int i=0;i<3;i++){
                System.out.println("Hello to the thread world");
                Thread.sleep(1000);
            }
        }catch(InterruptedException ie)
        {
            System.out.println(ie);
        }
    }
}
```

```
class demo1
{
    public static void main(String args[])
    {
        t1 ob1 = new t1();
    }
}
```

Output:

```
t1:Thread[Thread1,5,main]
Hello to the thread world
Hello to the thread world
Hello to the thread world
```

Multi-Threading – Creating Threads

- By Extending the Thread Class.

```
class t2 extends Thread{
    t2()
    {
        super();
        System.out.println("t2:"+this);
        start();
    }
    public void run(){
        try{
            for(int i=0;i<5;i++)
            {
                System.out.println("i:"+i);
                Thread.sleep(1000);
            }
        }catch(InterruptedException ie)
        {
            System.out.println(ie);
        }
    }
}
```

```
class demo1
{
    public static void main(String args[])
    {
        t1 ob1 = new t1();
    }
}
```

Output:

```
t1:Thread[Thread1,5,main]
Hello to the thread world
Hello to the thread world
Hello to the thread world
```

Multi-Threading – Creating Threads

- Multiple threads working in program

```
class t1 implements Runnable{
    Thread t;
    String name;
    t1() {
        t = new Thread(this);
        System.out.println("t:"+t);
        t.start();
    }
    public void run(){
        try{
            for(int i=0;i<3;i++){
                System.out.println("Hello to the thread world");
                Thread.sleep(1000);
            }
        }catch(InterruptedException ie)
        {
            System.out.println(ie);
        }
    }
}
```

```
class demo1
{
    public static void main(String args[])
    {
        data ob1 = new data("first");
        data ob2 = new data("second");
        data ob3 = new data("third");
    }
}
```

Output:

```
t:Thread[first,5,main]
t:Thread[second,5,main]
t:Thread[third,5,main]
first : 0
second : 0
third : 0
first : 1
second: 1
third : 1 . . . . .
```

Multi-Threading -

- Introduction
- Methods
- Main Thread
- Creating Threads
- **Synchronization**
- Thread Priorities

Multi-Threading – Synchronization

- When two or more threads need access to a shared resource, they need some way to ensure that the resource will be used by only one thread at a time. The process by which this is achieved is called synchronization.
- It can be used with the following concepts:
 - 1) With object of any class to achieve synchronization (**Synchronized block/statement**)
 - 2) With the inter threaded communication (**synchronized methods**)

Multi-Threading – Synchronization

- 1) With object of any class to achieve synchronization (**Synchronized block/statement**)

```
class data {  
    int a;  
    void fun(int x) {  
        a=x;  
        System.out.print("a = "+x);  
        try {  
            Thread.sleep(1000);  
        } catch(InterruptedException e) {  
            System.out.println("Interrupted");  
        }  
        System.out.println(" value");  
    }  
}
```

```
class get implements Runnable  
{  
    data obj;  
    Thread t;  
    int b;  
  
    get(data o1,int y)  
    {  
        obj = o1;  
        b=y;  
        t = new Thread(this);  
        t.start();  
    }  
    public void run()  
    {  
        synchronized(obj)  
        {  
            obj.fun(b);  
        }  
    }  
}
```

```
class sync  
{  
    public static void main(String args[])  
    {  
        data obj = new data();  
  
        get o1 = new get(obj,10);  
        get o2 = new get(obj,20);  
    }  
}
```

Multi-Threading – Synchronization (thread methods)

2) With the inter threaded communication (**synchronized methods**)

<u>Sr. No.</u>	<u>Method</u>	<u>Syntax</u>	<u>Details</u>
1	Thread Suspend	<code><obj_name>.<thread_obj>.suspend();</code> <code><thread_obj>.suspend();</code>	It is used to suspend the current / particular thread.
2	Thread Resume	<code><obj_name>.<thread_obj>.resume ();</code> <code><thread_obj>.resume();</code>	It is used to resume / start the particular thread which is suspended.
3	wait()	wait();	It tells the calling thread to give up the monitor / resource and go to sleep until some other thread enters the same monitor and calls notify(). - Used in try block
4	notify()	notify();	It wakes up the first thread that called wait() on the same object.
5	notifyAll()	notifyAll();	It wakes up all the threads that called wait() on the same object. The highest priority thread will run first.

Multi-Threading – Synchronization (thread methods)

2) With the inter threaded communication (**synchronized methods**)

- To avoid polling, Java includes an elegant inter-process communication mechanism via the wait(), notify(), and notifyAll() methods.
- Polling – Waiting for Resource/s (when more than one threads are running)

Multi-Threading – Synchronization (thread methods)

2) With the inter threaded communication (**synchronized methods**)

Steps to create program with Inter threaded communication:

- 1) At least four classes are created and they are for;
 - i) Class which is used as a resource
 - ii) Class which is used as a producer
 - iii) Class which is used as a consumer
 - iv) Class which contains main() function
- 2) Resource class contains two methods called get() and put()
 - i) **get()** : It is used get items to consumer
 - ii) **put()** : It is used to take the items from producer
- 3) Producer class produces items and pass it to the resource class through put().
- 4) Consumer class consumes the item from resource class through get().
- 5) Producer / consumer class creates a thread to do their task with the resource class.

Multi-Threading – Synchronization (thread methods)

2) With the inter threaded communication (**synchronized methods**)

Flow and Requirement of program:

- At first producer creates one item and call put() function through thread.
- Then put() function take that value and store it any variable.
- After that consumer calls get() function and take the value through thread.

Condition:

- When consumer call get() function and the new item is not available then its thread will be **waiting** until item will be available, once item is available then **notify** that thread.
- When producer call put() function and the last item is still available then its thread will be **waiting** until last item is consume by consumer, once last item is used then **notify** the producer's thread.
- **Resource class methods called “get()” and “put()”; both must synchronized method .**
- **Next program gives perfect inter threaded communication between producer and consumer from single Resource.**

Multi-Threading – Synchronization (thread methods)

2) With the inter threaded communication (**synchronized methods**)

RESOURCE **CLASS**

```
class data
{
    int a;
    boolean temp;
    synchronized void put(int n)
    {
        if(temp)
        {
            try
            {
                wait();
            } catch (InterruptedException e)
            {
                System.out.println("interrupted");
            }
        }
        a=n;
        System.out.println("put:"+a);
        temp=true;
        notify();
    }
}
```

```
synchronized int get()
{
    if(!temp)
    {
        try
        {
            wait();
        } catch (InterruptedException e)
        {
            System.out.println("interrupted");
        }
    }
    temp=false;
    notify();
    return a;
}
}
```

Multi-Threading – Synchronization (thread methods)

2) With the inter threaded communication (**synchronized methods**)

class p implements Runnable

```
{
    Thread t;
    data d;
    p(data d1)
    {
        d=d1;
        t = new Thread(this);
        t.start();
    }
    public void run()
    {
        int i=0;
        while(true)
        {
            d.put(i++);
        }
    }
}
```

class c implements Runnable

```
{
    Thread t;
    data d;
    c(data d1)
    {
        d=d1;
        t = new Thread(this);
        t.start();
    }
    public void run()
    {
        while(true)
        {
            System.out.println("get:"+d.get());
        }
    }
}
```

PRODUCER Class

&

CONSUMER Class

&

Main Class

class demo3

```
{
    public static void main(String a[])
    {
        data d = new data();
        p ob1 = new p(d);
        c ob2 = new c(d);
    }
}
```